In [116…
```python
import pandas as pd
import numpy as np
import seaborn as sns
import matplotlib.pyplot as plt

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.metrics import mean_squared_error, r2_score

import tensorflow as tf
from tensorflow.keras.models import Sequential
from tensorflow.keras.layers import Dense, Dropout
from tensorflow.keras.callbacks import EarlyStopping
```

In [101…
```python
# load the dataset
df = pd.read_csv('../data/boston_housing.csv')
df.head()
```

Out[101…

	CRIM	ZN	INDUS	CHAS	NOX	RM	AGE	DIS	RAD	TAX	PTRATIO
0	0.00632	18.0	2.31	0	0.538	6.575	65.2	4.0900	1	296	15.3
1	0.02731	0.0	7.07	0	0.469	6.421	78.9	4.9671	2	242	17.8
2	0.02729	0.0	7.07	0	0.469	7.185	61.1	4.9671	2	242	17.8
3	0.03237	0.0	2.18	0	0.458	6.998	45.8	6.0622	3	222	18.7
4	0.06905	0.0	2.18	0	0.458	7.147	54.2	6.0622	3	222	18.7

In [102…
```python
df.info()
```

```
<class 'pandas.DataFrame'>
RangeIndex: 506 entries, 0 to 505
Data columns (total 14 columns):
 #   Column   Non-Null Count  Dtype
---  ------   --------------  -----
 0   CRIM     506 non-null    float64
 1   ZN       506 non-null    float64
 2   INDUS    506 non-null    float64
 3   CHAS     506 non-null    int64
 4   NOX      506 non-null    float64
 5   RM       506 non-null    float64
 6   AGE      506 non-null    float64
 7   DIS      506 non-null    float64
 8   RAD      506 non-null    int64
 9   TAX      506 non-null    int64
 10  PTRATIO  506 non-null    float64
 11  B        506 non-null    float64
 12  LSTAT    506 non-null    float64
 13  MEDV     506 non-null    float64
dtypes: float64(11), int64(3)
memory usage: 55.5 KB
```

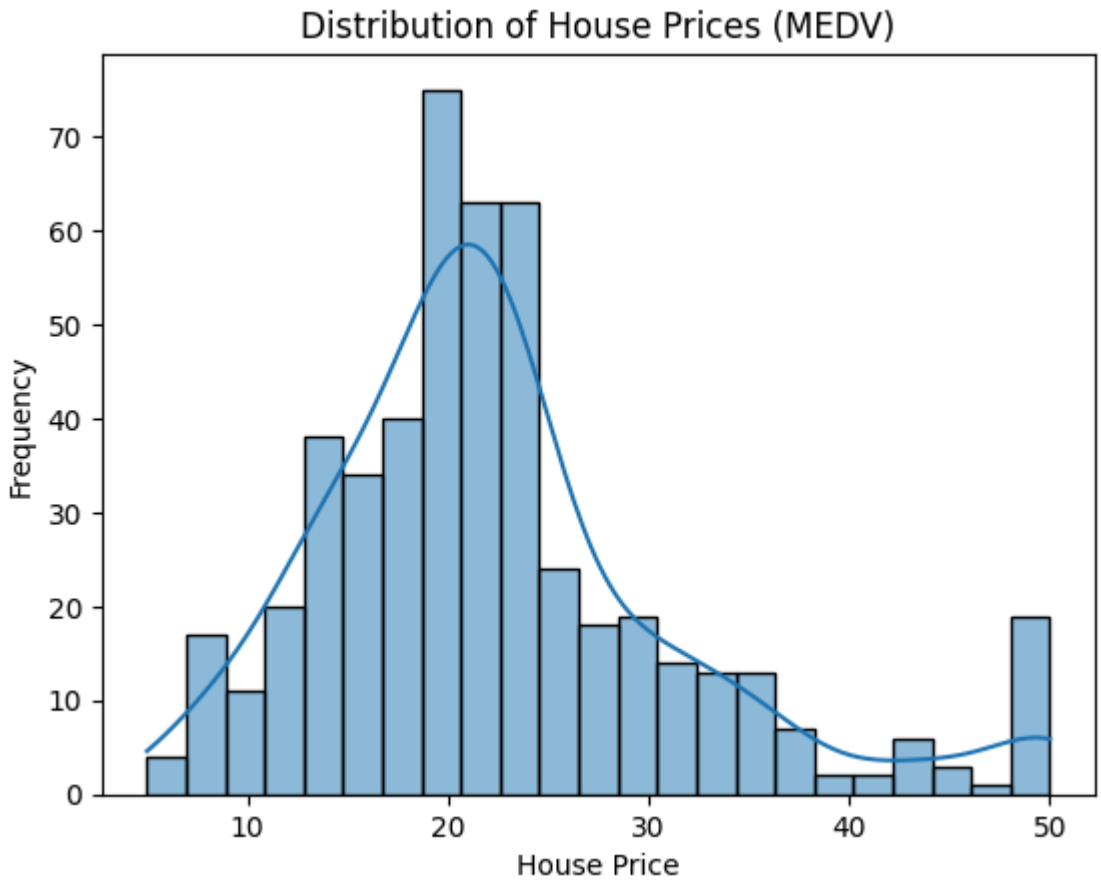
In [103…
```python
# null check.
df.isnull().sum()
```

Out[103…

```
CRIM       0
ZN         0
INDUS      0
CHAS       0
NOX        0
RM         0
AGE        0
DIS        0
RAD        0
TAX        0
PTRATIO    0
B          0
LSTAT      0
MEDV       0
dtype: int64
```

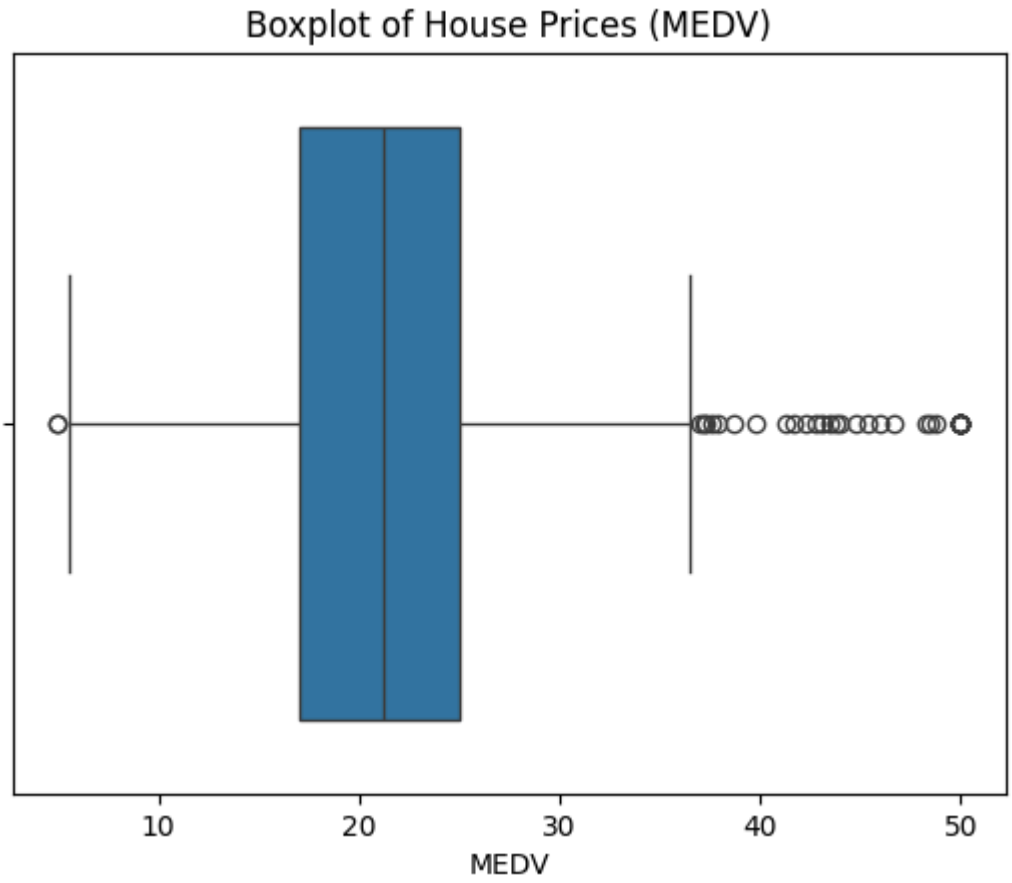
In [106…

```
# target variable distribution.

sns.histplot(data=df['MEDV'], kde=True)
plt.title('Distribution of House Prices (MEDV)')
plt.xlabel('House Price')
plt.ylabel('Frequency')
plt.show()
```



In [108…

```
# boxplot of the target variable.

sns.boxplot(x=df['MEDV'])
plt.title('Boxplot of House Prices (MEDV)')
plt.show()
```



In [109…
```
# removing the outliers.

Q1 = df['MEDV'].quantile(0.25)
Q3 = df['MEDV'].quantile(0.75)
IQR = Q3 - Q1

lower_bound = Q1 - 1.5 * IQR
upper_bound = Q3 + 1.5 * IQR

df = df[(df['MEDV'] >= lower_bound) & (df['MEDV'] <= upper_bound)]
```

In [111…
```
# split dataset features.
X = df.drop('MEDV', axis=1)
y = df['MEDV']
```

In [112…
```
# spit for train and test.
X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2,
```

In [113…
```
# scale the features.
scaler = StandardScaler()

X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
```

In [115…
```
# build and compile the neural network.

model = Sequential()

# input-layer + hidden-layer_1
model.add(Dense(64, activation='relu', input_shape=(X_train.shape[1],)))
model.add(Dropout(0.2))
```

```
# hidden-layer_2
model.add(Dense(32, activation='relu'))
model.add(Dropout(0.2))

# hidden-layer_3
model.add(Dense(16, activation='relu'))
model.add(Dropout(0.2))

# output-layer
model.add(Dense(1))

model.compile(
    optimizer='adam',
    loss='mse',
    metrics=['mae']
)
```

```
/Users/parimal/VisualStudioCodeProjects/dl-assignments/venv/lib/python3.1
3/site-packages/keras/src/layers/core/dense.py:107: UserWarning: Do not pa
ss an `input_shape`/`input_dim` argument to a layer. When using Sequential
models, prefer using an `Input(shape)` object as the first layer in the mo
del instead.
  super().__init__(activity_regularizer=activity_regularizer, **kwargs)
```

In [118…

```
# create early stopping.
early_stop = EarlyStopping(
    monitor='val_loss',
    patience=15,
    restore_best_weights=True
)
```

In [119…

```
# train the model.

history = model.fit(
    X_train,
    y_train,
    validation_split=0.1,
    epochs=100,
    verbose=1,
    batch_size=16,
    callbacks=[early_stop]
)
```

```
Epoch 1/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 1s 5ms/step - loss: 467.2620 - mae: 20.6647 - v
al_loss: 434.4172 - val_mae: 19.7598
Epoch 2/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 419.0316 - mae: 19.4471 - v
al_loss: 372.8651 - val_mae: 18.2053
Epoch 3/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 335.1677 - mae: 17.1405 - v
al_loss: 246.4107 - val_mae: 14.5157
Epoch 4/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 191.1353 - mae: 12.2204 - v
al_loss: 85.4325 - val_mae: 7.9087
Epoch 5/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 86.6748 - mae: 7.7035 - val
_loss: 33.4319 - val_mae: 4.7810
Epoch 6/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 67.4123 - mae: 6.6792 - val
_loss: 22.1097 - val_mae: 3.8185
Epoch 7/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 55.1288 - mae: 5.8936 - val
_loss: 16.1649 - val_mae: 3.1646
Epoch 8/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 51.2946 - mae: 5.6418 - val
_loss: 16.1980 - val_mae: 3.2116
Epoch 9/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 46.2047 - mae: 5.4283 - val
_loss: 19.6965 - val_mae: 3.5811
Epoch 10/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 47.1718 - mae: 5.4165 - val
_loss: 17.1128 - val_mae: 3.2237
Epoch 11/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 54.9826 - mae: 5.8469 - val
_loss: 18.7816 - val_mae: 3.3960
Epoch 12/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 39.7073 - mae: 4.8856 - val
_loss: 16.4673 - val_mae: 3.1446
Epoch 13/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 41.0931 - mae: 5.0579 - val
_loss: 15.6583 - val_mae: 3.0671
Epoch 14/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 40.3673 - mae: 5.1335 - val
_loss: 14.3557 - val_mae: 2.8622
Epoch 15/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 44.5309 - mae: 5.1065 - val
_loss: 17.3951 - val_mae: 3.2194
Epoch 16/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 39.4182 - mae: 4.8849 - val
_loss: 15.6441 - val_mae: 3.0371
Epoch 17/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 38.1139 - mae: 5.0164 - val
_loss: 15.4334 - val_mae: 3.0909
Epoch 18/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 36.8355 - mae: 4.8659 - val
_loss: 14.5121 - val_mae: 2.9716
Epoch 19/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 31.2762 - mae: 4.4485 - val
_loss: 15.1718 - val_mae: 2.9993
Epoch 20/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 32.0658 - mae: 4.4822 - val
_loss: 12.6944 - val_mae: 2.6889
```

```
Epoch 21/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 33.1880 - mae: 4.5161 - val
_loss: 14.4943 - val_mae: 2.9896
Epoch 22/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 37.2368 - mae: 4.7693 - val
_loss: 12.9745 - val_mae: 2.7598
Epoch 23/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 34.2680 - mae: 4.6145 - val
_loss: 13.0339 - val_mae: 2.7725
Epoch 24/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 34.5277 - mae: 4.6150 - val
_loss: 16.5191 - val_mae: 3.2199
Epoch 25/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 35.2965 - mae: 4.7051 - val
_loss: 14.2809 - val_mae: 2.8951
Epoch 26/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 33.6058 - mae: 4.5336 - val
_loss: 12.7504 - val_mae: 2.7689
Epoch 27/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 35.5085 - mae: 4.6387 - val
_loss: 11.9825 - val_mae: 2.6718
Epoch 28/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 35.9705 - mae: 4.6232 - val
_loss: 13.6912 - val_mae: 2.9298
Epoch 29/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 33.8283 - mae: 4.4794 - val
_loss: 11.7075 - val_mae: 2.5727
Epoch 30/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 32.2046 - mae: 4.4850 - val
_loss: 12.7440 - val_mae: 2.7657
Epoch 31/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 30.4683 - mae: 4.3290 - val
_loss: 12.6361 - val_mae: 2.7363
Epoch 32/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 29.4268 - mae: 4.3201 - val
_loss: 11.2538 - val_mae: 2.5058
Epoch 33/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 33.1757 - mae: 4.4133 - val
_loss: 12.4413 - val_mae: 2.7335
Epoch 34/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 35.9024 - mae: 4.6156 - val
_loss: 11.7122 - val_mae: 2.6227
Epoch 35/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 30.4682 - mae: 4.2933 - val
_loss: 12.1325 - val_mae: 2.7090
Epoch 36/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 35.1473 - mae: 4.5555 - val
_loss: 12.7838 - val_mae: 2.7844
Epoch 37/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 4ms/step - loss: 35.0219 - mae: 4.5369 - val
_loss: 10.8869 - val_mae: 2.3782
Epoch 38/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 30.7234 - mae: 4.3429 - val
_loss: 12.2341 - val_mae: 2.7164
Epoch 39/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 33.1465 - mae: 4.4261 - val
_loss: 12.6969 - val_mae: 2.7886
Epoch 40/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 32.2782 - mae: 4.4811 - val
_loss: 12.6536 - val_mae: 2.7652
```

```
Epoch 41/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 34.5198 - mae: 4.6461 - val
_loss: 11.6049 - val_mae: 2.5454
Epoch 42/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 33.0996 - mae: 4.2568 - val
_loss: 11.6636 - val_mae: 2.5841
Epoch 43/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 35.2499 - mae: 4.5470 - val
_loss: 11.9704 - val_mae: 2.6504
Epoch 44/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 33.4445 - mae: 4.5322 - val
_loss: 10.9768 - val_mae: 2.4557
Epoch 45/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 37.0298 - mae: 4.7169 - val
_loss: 11.1566 - val_mae: 2.4975
Epoch 46/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 34.1587 - mae: 4.5405 - val
_loss: 15.6953 - val_mae: 3.1525
Epoch 47/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 32.0443 - mae: 4.5431 - val
_loss: 11.6452 - val_mae: 2.6107
Epoch 48/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 32.8714 - mae: 4.2982 - val
_loss: 9.9639 - val_mae: 2.2705
Epoch 49/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 2ms/step - loss: 33.1210 - mae: 4.5758 - val
_loss: 11.0487 - val_mae: 2.4833
Epoch 50/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 31.8927 - mae: 4.4376 - val
_loss: 10.3072 - val_mae: 2.3321
Epoch 51/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 34.3686 - mae: 4.4182 - val
_loss: 12.1334 - val_mae: 2.7101
Epoch 52/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 32.6034 - mae: 4.3743 - val
_loss: 10.7047 - val_mae: 2.5150
Epoch 53/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 30.5688 - mae: 4.3124 - val
_loss: 10.3998 - val_mae: 2.3621
Epoch 54/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 31.3198 - mae: 4.3454 - val
_loss: 10.2562 - val_mae: 2.3228
Epoch 55/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 28.6355 - mae: 4.3121 - val
_loss: 11.2069 - val_mae: 2.5223
Epoch 56/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 32.8503 - mae: 4.4543 - val
_loss: 11.2983 - val_mae: 2.5294
Epoch 57/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 33.9456 - mae: 4.6735 - val
_loss: 11.1409 - val_mae: 2.5064
Epoch 58/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 31.6028 - mae: 4.4411 - val
_loss: 12.4577 - val_mae: 2.7926
Epoch 59/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 27.2420 - mae: 4.1852 - val
_loss: 11.2438 - val_mae: 2.5637
Epoch 60/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 30.8395 - mae: 4.3662 - val
_loss: 10.3846 - val_mae: 2.3727
```

```
Epoch 61/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 30.8522 - mae: 4.3425 - val
_loss: 9.6559 - val_mae: 2.1446
Epoch 62/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 31.4604 - mae: 4.3314 - val
_loss: 10.9822 - val_mae: 2.4588
Epoch 63/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 27.7086 - mae: 4.0496 - val
_loss: 12.3121 - val_mae: 2.7717
Epoch 64/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 27.1686 - mae: 3.9778 - val
_loss: 10.4551 - val_mae: 2.3823
Epoch 65/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 31.5618 - mae: 4.2619 - val
_loss: 11.2998 - val_mae: 2.5601
Epoch 66/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 25.2603 - mae: 4.0786 - val
_loss: 10.9230 - val_mae: 2.4471
Epoch 67/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 29.8912 - mae: 4.2800 - val
_loss: 10.8084 - val_mae: 2.4720
Epoch 68/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 24.0815 - mae: 3.8583 - val
_loss: 10.5390 - val_mae: 2.3581
Epoch 69/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 31.6194 - mae: 4.1920 - val
_loss: 10.1817 - val_mae: 2.2604
Epoch 70/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 3ms/step - loss: 28.2897 - mae: 4.1921 - val
_loss: 10.6596 - val_mae: 2.4756
Epoch 71/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 33.2498 - mae: 4.3748 - val
_loss: 10.3508 - val_mae: 2.4117
Epoch 72/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 28.2643 - mae: 4.1614 - val
_loss: 10.5072 - val_mae: 2.4267
Epoch 73/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 33.7273 - mae: 4.6203 - val
_loss: 10.0863 - val_mae: 2.2730
Epoch 74/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 30.8101 - mae: 4.1731 - val
_loss: 12.2987 - val_mae: 2.7310
Epoch 75/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 27.0400 - mae: 4.0139 - val
_loss: 10.7132 - val_mae: 2.4237
Epoch 76/100
21/21 ━━━━━━━━━━━━━━━━━━━━ 0s 1ms/step - loss: 28.6607 - mae: 4.2022 - val
_loss: 11.5004 - val_mae: 2.6044
```

In [120…

```
# model prediction.
y_pred = model.predict(X_test)
```

```
3/3 ━━━━━━━━━━━━━━━━━━━━ 0s 11ms/step
```

In [121…

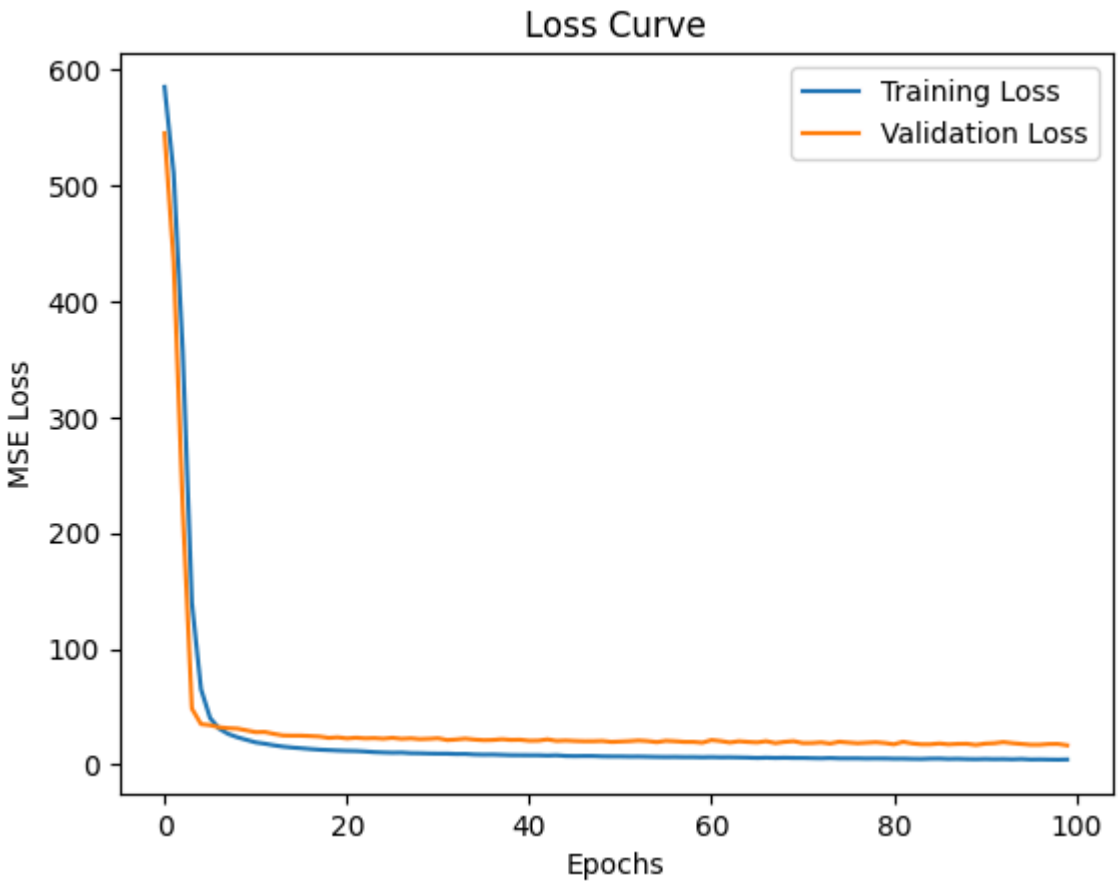
```
# model evaluation.

mse = mean_squared_error(y_test, y_pred)
rmse = np.sqrt(mse)
r2 = r2_score(y_test, y_pred)
```

```
print("\nModel Performance:")
print("Mean Squared Error:", mse)
print("Root Mean Squared Error:", rmse)
print("R2 Score:", r2)
```

```
Model Performance:
Mean Squared Error: 7.167092137141366
Root Mean Squared Error: 2.6771425320930087
R2 Score: 0.82133840097201
```

In [98]:

```
# plot training loss.

plt.plot(history.history['loss'], label='Training Loss')
plt.plot(history.history['val_loss'], label='Validation Loss')
plt.title('Loss Curve')
plt.xlabel('Epochs')
plt.ylabel('MSE Loss')
plt.legend()
plt.show()
```



In [123…

```
# actual Vs predicted plot.

plt.scatter(y_test, y_pred)
plt.xlabel("Actual Prices")
plt.ylabel("Predicted Prices")
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'r--
plt.title("Actual vs Predicted House Prices")
plt.show()
```

Actual vs Predicted House Prices
Predicted Prices
Actual Prices
5
10
15
20
25
30
35

In [ ]: